Parcial 2 – citas veterinarias

Integrante

Samuel Iván Heredia Moy

Docente:

Carlos Carrascal

Programación de computadores II

Universidad de Santander

Facultad de ingeniería de software

14 de octubre de 2026

Análisis

El software atiende a clínicas veterinarias que lo contratan. Cada clínica maneja su propio grupo de veterinarios, su catálogo de servicios, sus propietarios y sus mascotas, y sobre esos datos programa, cancela y atiende citas. Del enunciado salen nueve clases: Municipio, Clinica, Veterinario, Servicio, Propietario, Mascota, Cita, Factura y LineaFactura. El estado de una cita (Programada, Cancelada o Atendida) se modela con la enumeración EstadoCita.

El enunciado fija dónde es único cada identificador. El NIT identifica a la clínica en toda la aplicación. La cédula de un propietario y el código de un servicio solo son únicos dentro de una clínica (regla 7), así que esas colecciones pertenecen a cada objeto Clinica y no a un repositorio global. Los códigos de cita y de factura (CIT-001, FAC-001) son únicos en toda la aplicación: en los ejemplos, la cita de la clínica Patitas Felices es CIT-003 y viene después de dos citas de Huellitas.

La arquitectura tiene tres capas. La capa de presentación lee datos por consola y muestra mensajes. La capa de lógica contiene las entidades, las reglas de negocio y los servicios que coordinan cada funcionalidad. La capa de datos guarda las colecciones en diccionarios en memoria y responde consultas. La clase Main crea los objetos de las tres capas y los conecta.

Supuestos del diseño

El enunciado deja algunos puntos abiertos. Estas son las decisiones tomadas para cubrirlos.

1. Al programar una cita, quien opera el sistema indica la cédula del veterinario. El caso 2 lo exige: la solicitud se rechaza porque se eligió a una veterinaria ocupada, aunque otro veterinario de la clínica estaba libre.
2. Cancelar una cita recibe la fecha actual como dato de entrada, porque los ejemplos ocurren en noviembre de 2026 y no pueden depender del reloj del equipo. La cancelación se permite si la fecha actual es anterior a la fecha de la cita (regla 4).
3. Al cancelar se valida primero el estado y después la fecha. El caso 4 se rechaza por estar la cita atendida, sin importar la fecha.
4. La fecha de la factura es la fecha de la cita, porque en el caso 1 la atención y la factura ocurren el mismo día.
5. Para los propietarios el enunciado da municipio y departamento, sin código DANE. Municipio acepta un código DANE vacío (None). Para las clínicas el código es obligatorio.
6. Una mascota se identifica dentro de una clínica por la cédula de su propietario y su nombre, porque el enunciado no define otro identificador. La cédula del veterinario se asume única dentro de su clínica.
7. Validaciones de datos: textos no vacíos, tarifa y peso mayores que cero, año de nacimiento entero.
8. Los códigos se generan en secuencia con formato CIT-001 y FAC-001, con un contador en el repositorio correspondiente.
9. Las citas de una clínica en una fecha y el historial de una mascota se devuelven ordenados por fecha y hora.

10. La opción de cargar datos de ejemplo registra las dos clínicas con sus veterinarios y catálogos. Los propietarios, las mascotas y las citas se ingresan por el menú al ejecutar los casos.
11. Los diagramas de diseño marcan los atributos privados con el signo menos. En Python se codifican con guion bajo inicial y se exponen con @property.

Reglas de negocio y dónde se validan

Regla	Dónde se valida
1. Estados de la cita; al crearse queda Programada	Constructor de Cita y enumeración EstadoCita
2. Un veterinario no tiene dos citas programadas en la misma fecha y hora	ServicioCitas.programar_cita, con RepositorioCitas.buscar_programada_veterinario
3. Una mascota no tiene dos citas programadas en la misma fecha y hora	ServicioCitas.programar_cita, con RepositorioCitas.buscar_programada_mascota
4. Solo se cancela una cita Programada y antes de su fecha; las canceladas se conservan	Cita.cancelar; ningún repositorio elimina citas
5. Solo se atiende una cita Programada; al atenderla se genera la factura	Cita.atender y ServicioCitas.atender_cita
6. La factura tiene número único, fecha, datos, detalle y total igual a la suma de las tarifas	RepositorioFacturas.siguiente_numero y Factura.valor_total
7. NIT único; cédula de propietario y código de servicio únicos por clínica	RepositorioClinicas.agregar, Clinica.agregar_propietario y Clinica.agregar_servicio
8. Ante una regla incumplida se informa el motivo y el programa continúa	MenuConsola captura ReglaNegocioError y vuelve al menú

Diagrama de dominio parcial

Muestra las clases del dominio y sus asociaciones con multiplicidad, sin atributos.

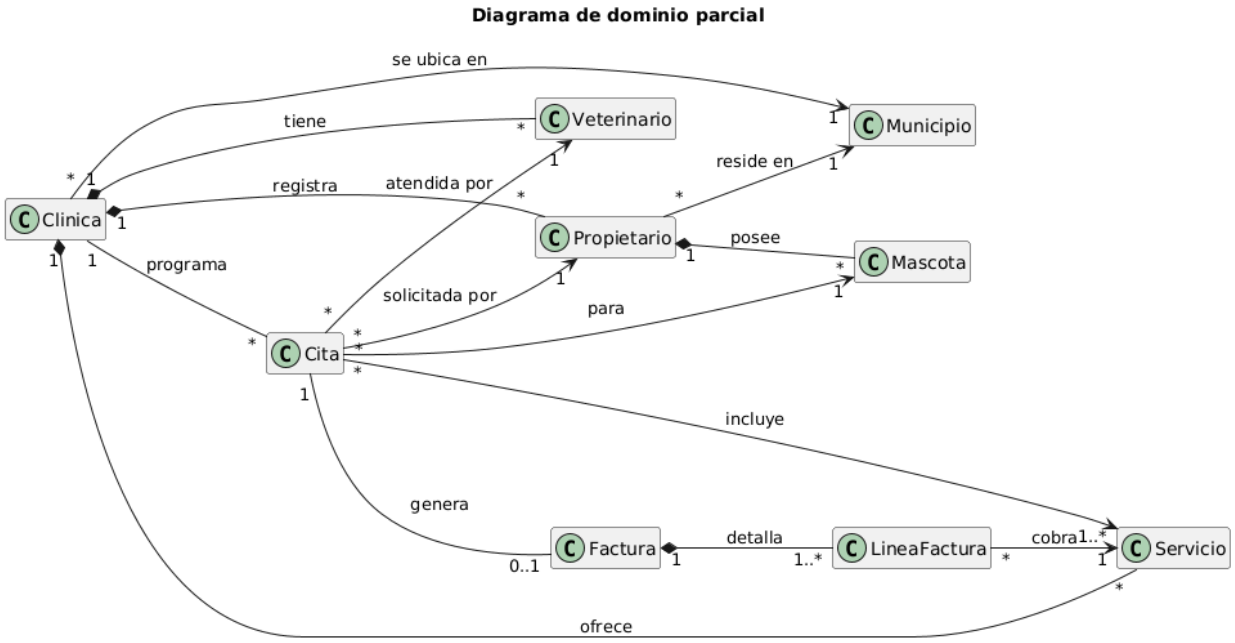


Diagrama de dominio

Agrega los atributos de cada clase del diagrama anterior, con su tipo.

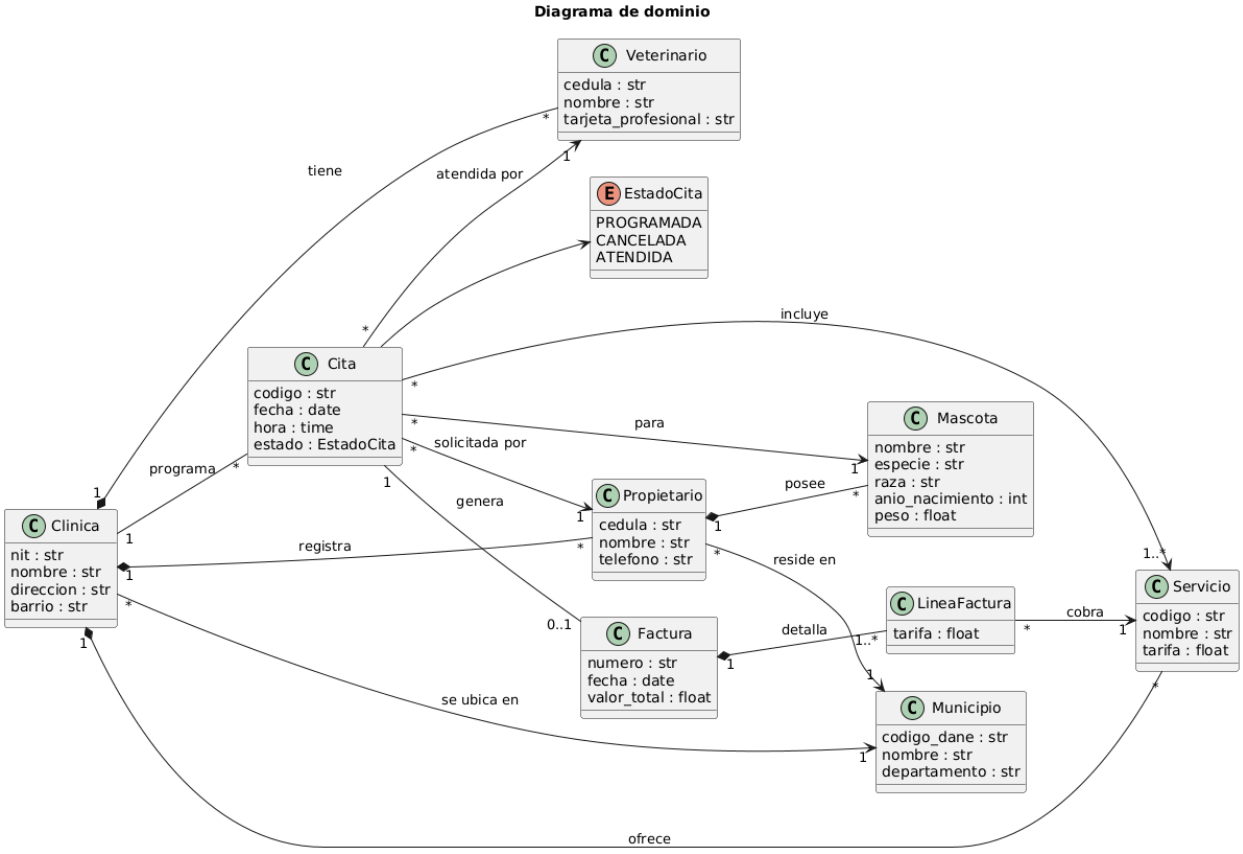
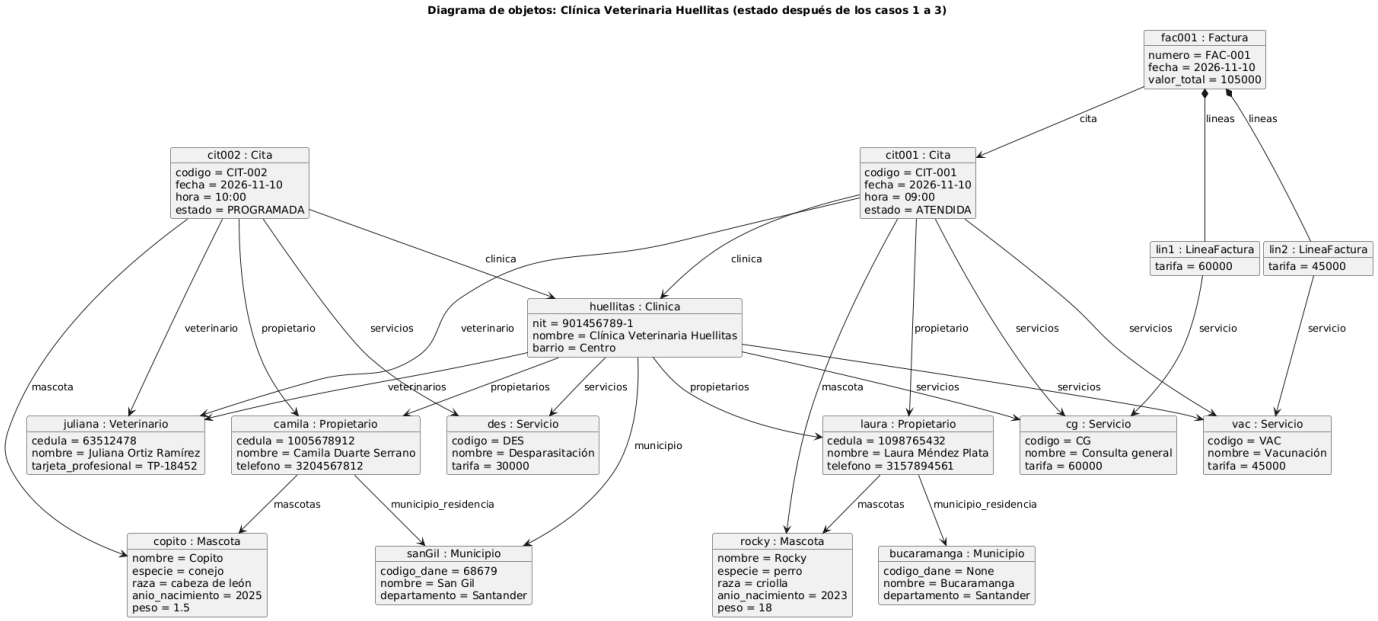


Diagrama de objetos

Clínica Veterinaria Huellitas:



Clinica veterinaria patitas felices

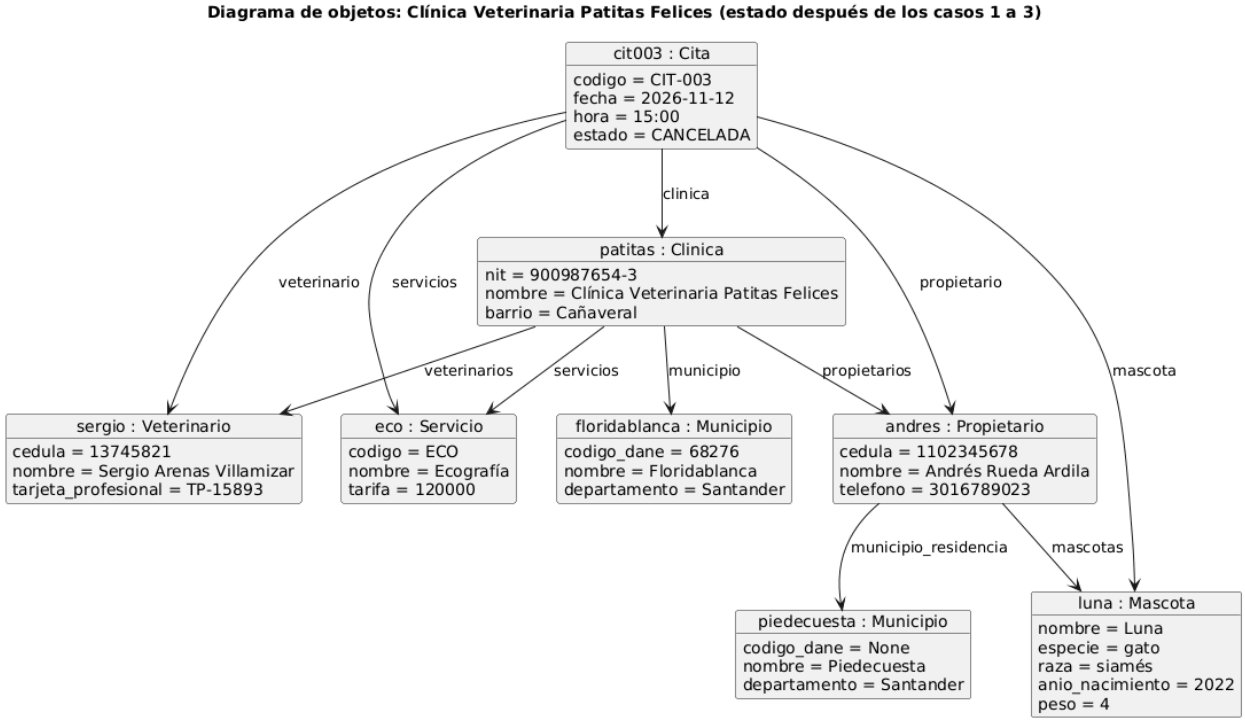
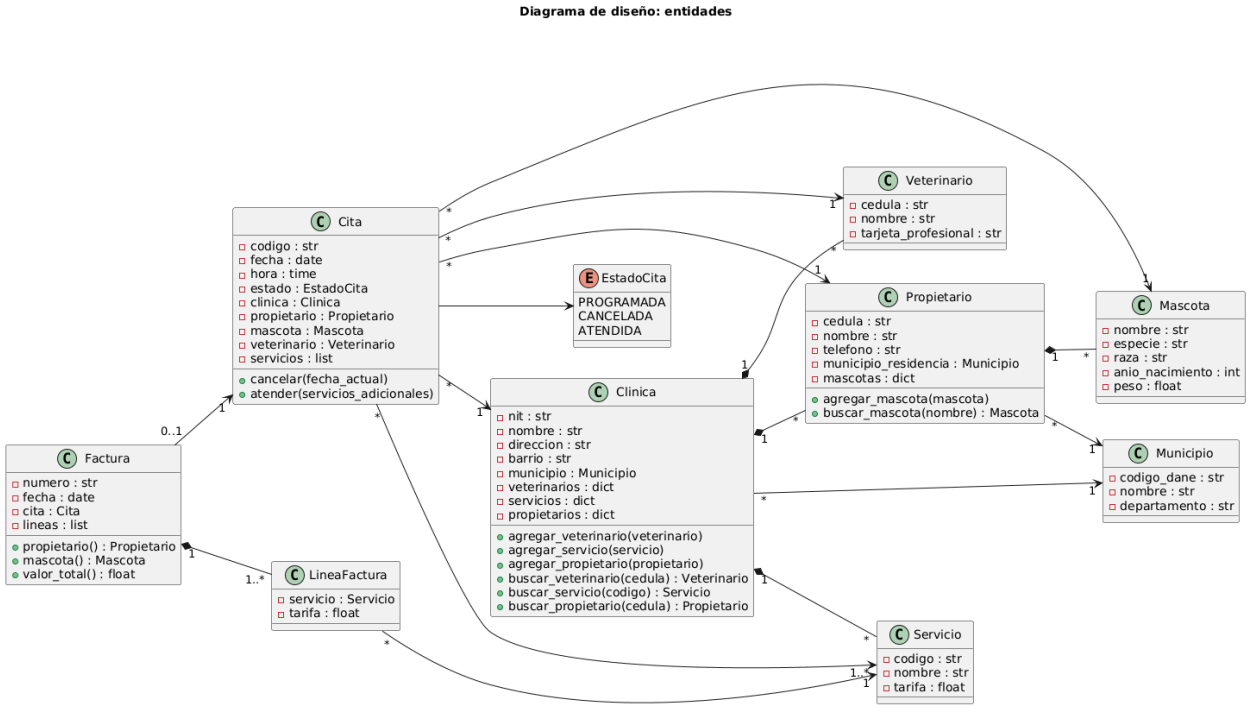


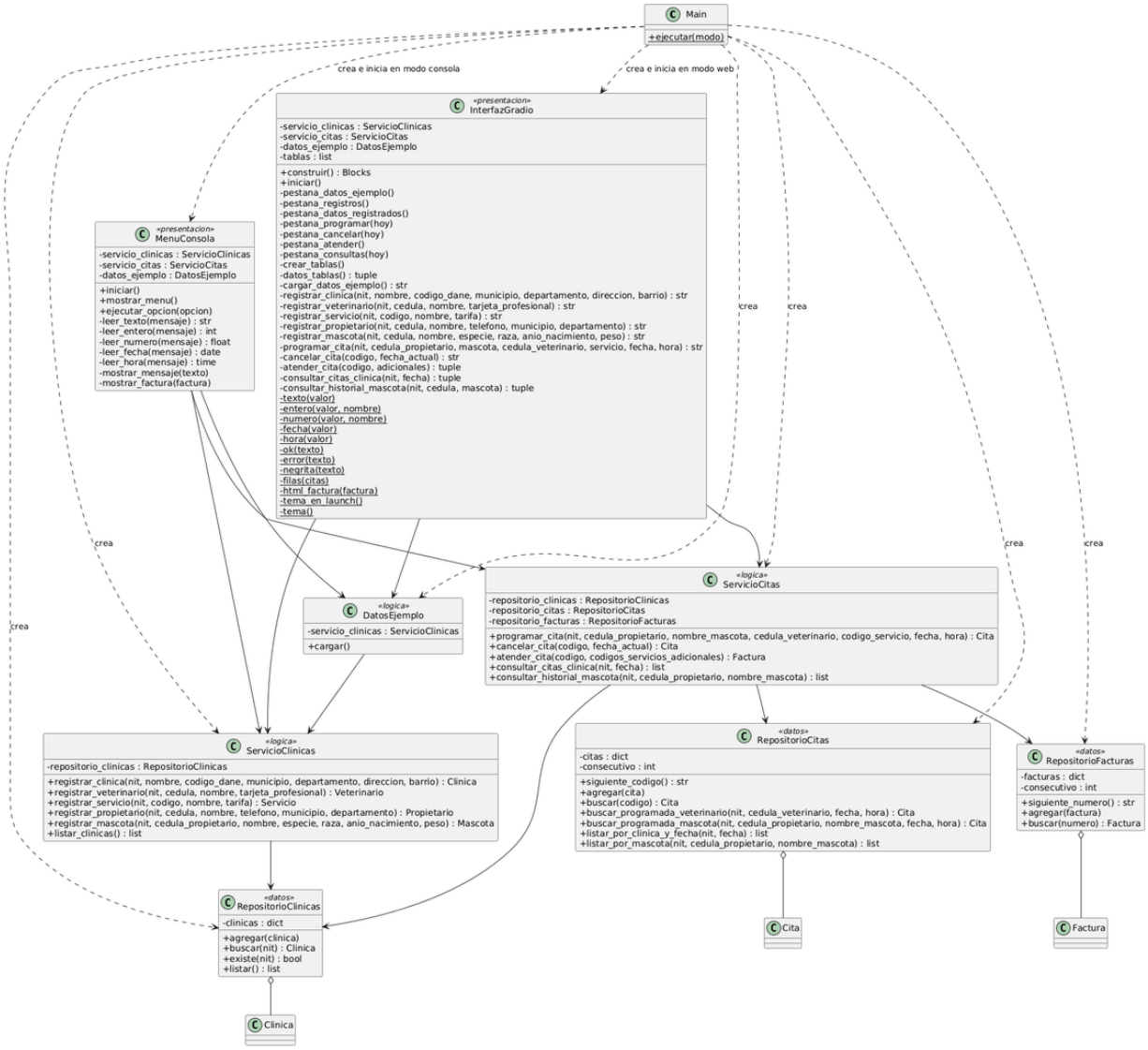
Diagrama de diseño

El diagrama de diseño se divide en tres figuras para que cada una se pueda leer: las entidades, las tres capas con sus servicios y repositorios, y la jerarquía de excepciones. Las tres forman un solo diseño.

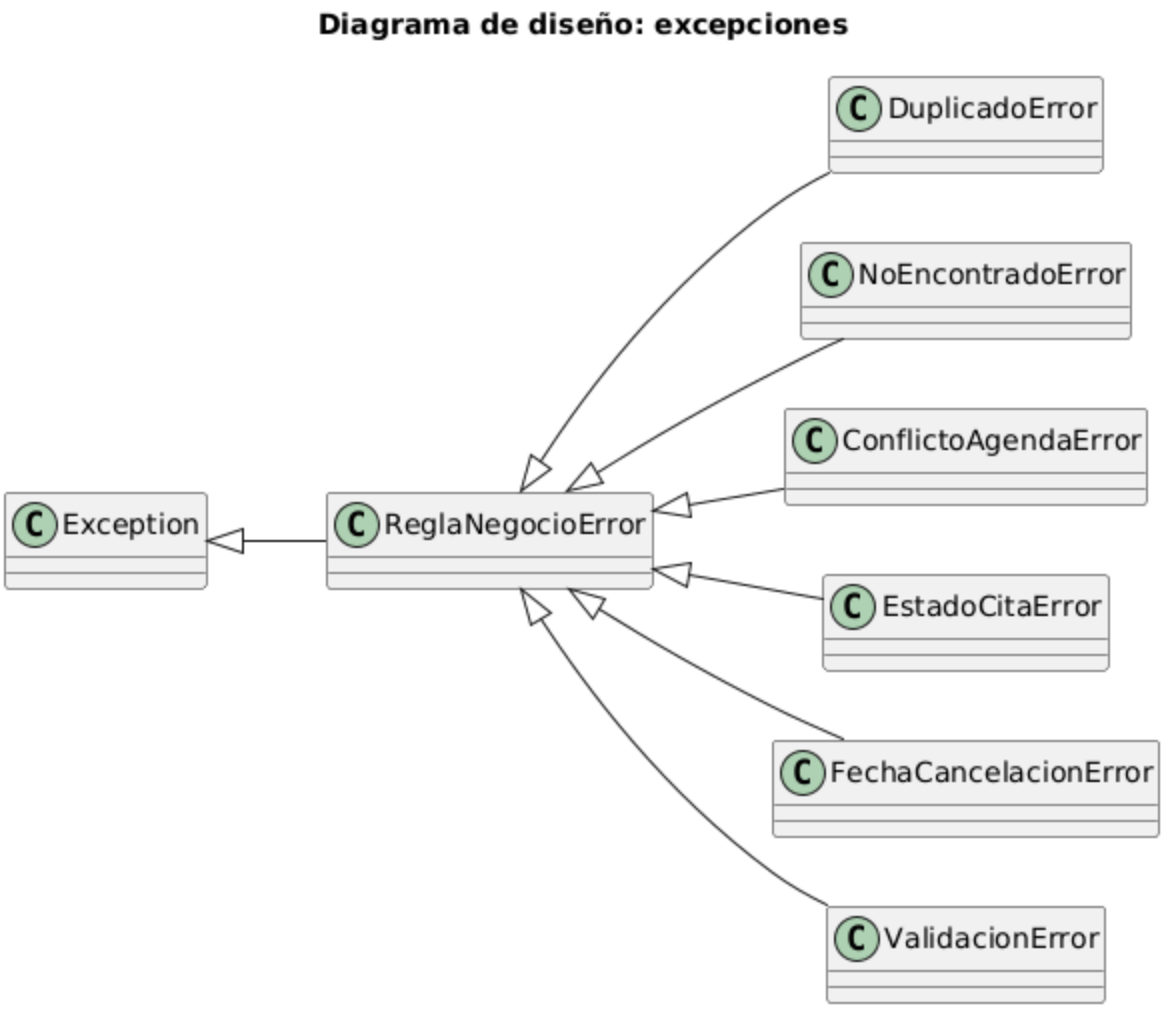
Entidades



Capas: presentación, lógica y datos



Excepciones



Estructura de paquetes

```
main.py                     clase Main
requirements.txt            biblioteca gradio
presentacion/
    menu_consola.py         MenuConsola
    interfaz_gradio.py      InterfazGradio
logica/
    municipio.py, clinica.py, veterinario.py, servicio.py,
    propietario.py, mascota.py, estado_cita.py, cita.py,
    factura.py, linea_factura.py
    servicio_clinicas.py, servicio_citas.py, datos_ejemplo.py
    excepciones.py
datos/
    repositorio_clinicas.py, repositorio_citas.py,
    repositorio_facturas.py
```

Explicación de las clases

Entidades

Todas las entidades tienen atributos privados y propiedades de solo lectura, salvo donde se indica. Las validaciones de datos se hacen en el constructor y lanzan ValidacionError.

Municipio guarda codigo_dane (str, vacío para los municipios de residencia de los propietarios), nombre (str) y departamento (str). No tiene métodos.

Clinica representa a la clínica que contrata el software. Guarda nit, nombre, direccion y barrio (todos str), municipio (Municipio) y tres diccionarios privados: veterinarios con la cédula como llave, servicios con el código y propietarios con la cédula. Los métodos agregar_veterinario, agregar_servicio y agregar_propietario insertan en el diccionario correspondiente y lanzan DuplicadoError si la llave ya existe (regla 7). Los métodos buscar_veterinario, buscar_servicio y buscar_propietario devuelven el objeto o lanzan NoEncontradoError. Las propiedades veterinarias, servicios y propietarios devuelven listas de solo lectura.

Veterinario guarda cedula, nombre y tarjeta_profesional (todos str). Servicio guarda codigo y nombre (str) y tarifa (float, mayor que cero). Ninguna de las dos clases tiene métodos.

Propietario guarda cedula, nombre y telefono (str), municipio_residencia (Municipio) y el diccionario privado mascotas con el nombre como llave. agregar_mascota lanza DuplicadoError si el propietario ya tiene una mascota con ese nombre. buscar_mascota devuelve la mascota o lanza NoEncontradoError.

Mascota guarda nombre, especie y raza (str), anio_nacimiento (int, año aproximado) y peso (float, en kilogramos, mayor que cero).

EstadoCita es una enumeración con los valores PROGRAMADA, CANCELADA y ATENDIDA.

Cita guarda codigo (str), fecha (date), hora (time), estado (EstadoCita), las referencias clinica, propietario, mascota y veterinario, y la lista servicios, que empieza con el servicio solicitado. El constructor asigna el estado PROGRAMADA (regla 1). El método cancelar(fecha_actual) lanza EstadoCitaError si el estado no es PROGRAMADA y FechaCancelacionError si fecha_actual no es anterior a fecha. Si no hay error, cambia el estado a CANCELADA (regla 4). El método atender(servicios_adicionales) lanza EstadoCitaError si el estado no es PROGRAMADA. En caso contrario agrega los servicios a la lista y cambia el estado a ATENDIDA (regla 5).

LineaFactura guarda el servicio prestado y la tarifa vigente al momento de facturar. Copiar la tarifa conserva el valor facturado aunque el catálogo cambie después.

Factura guarda numero (str), fecha (date), la cita que se factura y la lista lineas. Las propiedades propietario y mascota se obtienen de la cita. valor_total devuelve la suma de las tarifas de las líneas (regla 6).

Capa de datos

Los tres repositorios guardan sus objetos en diccionarios en memoria.

RepositorioClinicas guarda las clínicas con el NIT como llave. agregar lanza DuplicadoError si el NIT ya existe (regla 7). buscar lanza NoEncontradoError si no encuentra la clínica. existe devuelve un bool y listar devuelve todas las clínicas.

RepositorioCitas guarda las citas con su código como llave y mantiene un contador consecutivo. siguiente_codigo devuelve CIT-001, CIT-002 y así en adelante. buscar_programada_veterinario y buscar_programada_mascota devuelven la cita programada que coincida con clínica, persona, fecha y hora, o None si no hay. listar_por_clinica_y_fecha y listar_por_mascota devuelven listas ordenadas por fecha y hora. Ningún método elimina citas, de modo que las canceladas quedan en el historial (regla 4).

RepositorioFacturas guarda las facturas con su número como llave y su propio contador. siguiente_numero devuelve FAC-001, FAC-002 y así en adelante. agregar y buscar completan la clase.

Capa de lógica

ServicioClinicas coordina los registros de la clínica y de lo que contiene. registrar_clinica crea el Municipio y la Clinica y los guarda en RepositorioClinicas. registrar_veterinario, registrar_servicio, registrar_propietario y registrar_mascota buscan la clínica (y el propietario, en el caso de la mascota) y delegan la inserción en los métodos agregar de las entidades.

ServicioCitas coordina las funcionalidades 4 a 8 del enunciado. programar_cita busca la clínica, el propietario, la mascota, el veterinario y el servicio, verifica las reglas 2 y 3 con el repositorio de citas, crea la Cita con el código siguiente y la guarda. cancelar_cita busca la cita y llama a Cita.cancelar. atender_cita busca la cita y los servicios adicionales en el catálogo de su clínica, llama a Cita.atender, crea una LineaFactura por cada servicio de la cita, crea la Factura con el número siguiente y la guarda. consultar_citas_clinica y consultar_historial_mascota delegan en las consultas de RepositorioCitas.

DatosEjemplo registra, usando ServicioClinicas, las dos clínicas del enunciado con sus veterinarios y sus catálogos de servicios.

Las excepciones heredan de ReglaNegocioError. DuplicadoError cubre los identificadores repetidos (regla 7). NoEncontradoError cubre las búsquedas sin resultado. ConflictoAgendaError cubre las reglas 2 y 3. EstadoCitaError cubre los intentos de cancelar o atender una cita que no está Programada. FechaCancelacionError cubre la cancelación que no es anterior a la fecha de la cita. ValidacionError cubre los datos inválidos.

Capa de presentación y Main

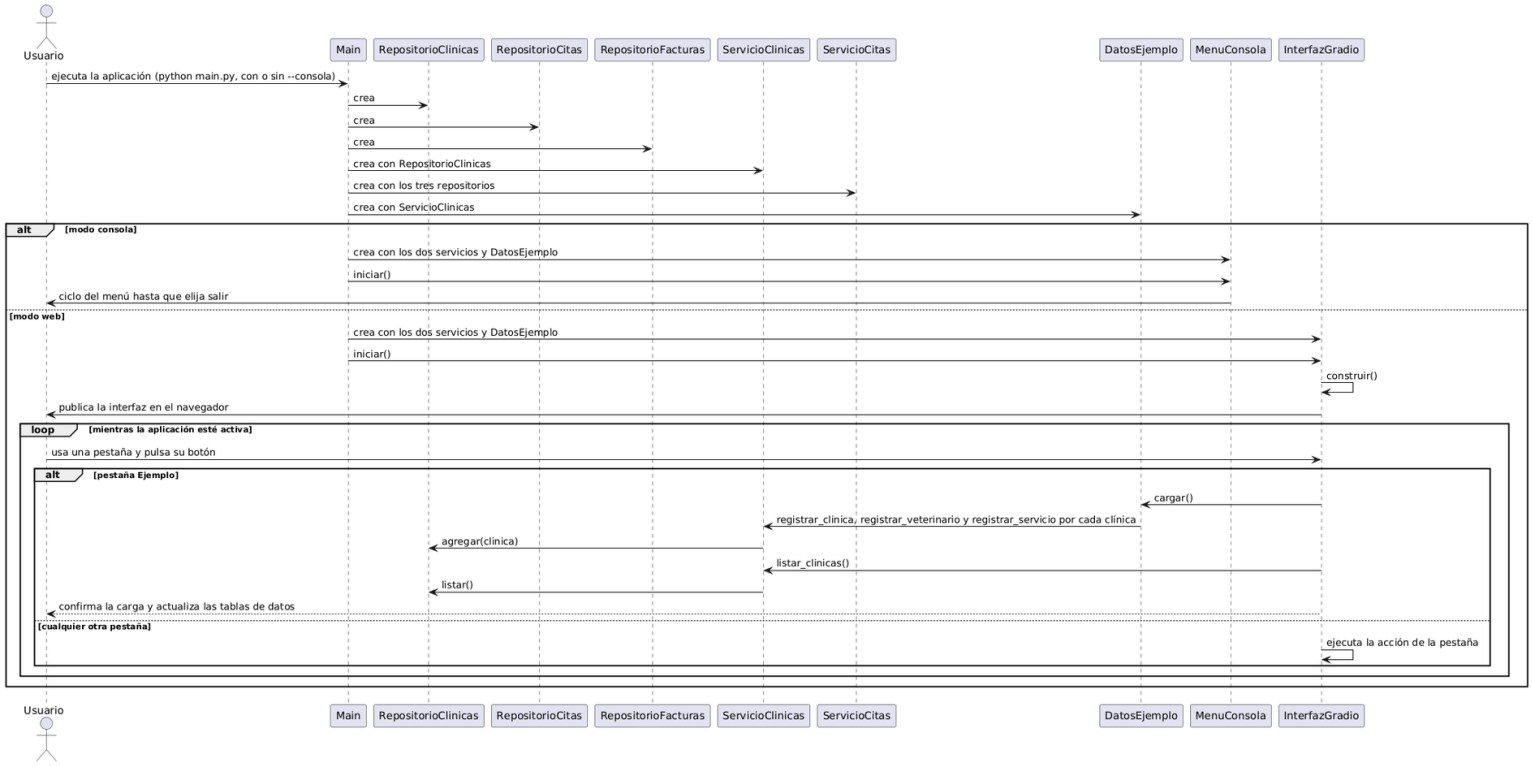
MenuConsola ofrece una opción por cada funcionalidad del enunciado (registrar clínica, registrar veterinarios y catálogo, registrar propietario y mascotas, programar, cancelar, atender y facturar, consultar citas de una clínica en una fecha, consultar historial de una mascota), una opción para cargar los datos de ejemplo y una para salir. Cada opción es un método privado que pide los datos con leer_texto, leer_fecha o leer_hora, llama al servicio y muestra el resultado. ejecutar_opcion captura ReglaNegocioError, muestra el motivo con mostrar_mensaje y vuelve al menú, de modo que el programa no se detiene (regla 8).

Main tiene el método estático ejecutar, que crea los repositorios, los servicios, DatosEjemplo y MenuConsola, y llama a MenuConsola.iniciar.

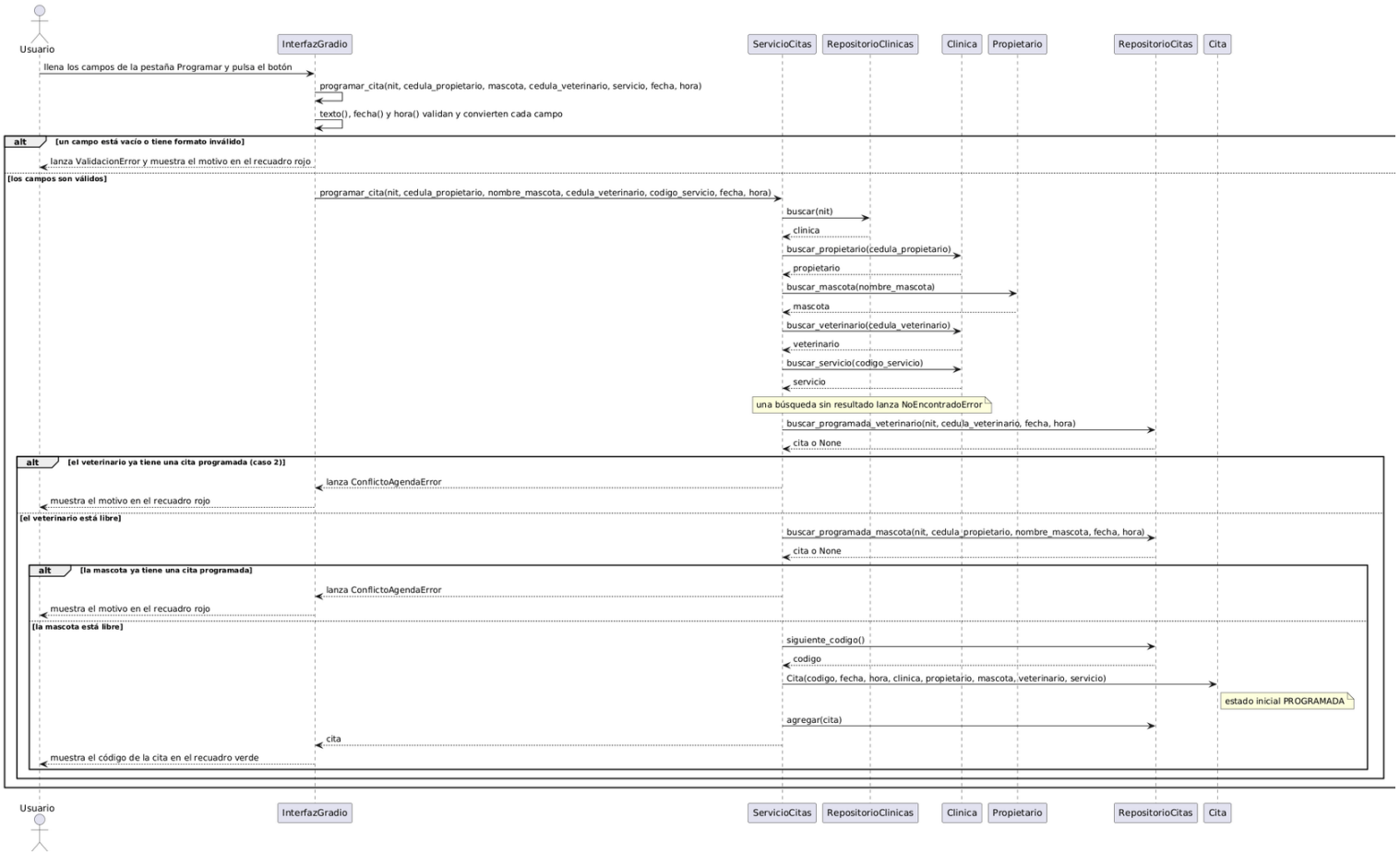
Diagramas de secuencia

El enunciado pide secuencias que cubran la capa de presentación, la de lógica, la de datos y la clase Main, como mínimo para programar una cita, cancelarla y atenderla con su factura. Se dibuja una secuencia de arranque en la que actúa Main y una secuencia por cada funcionalidad, con participantes de las tres capas.

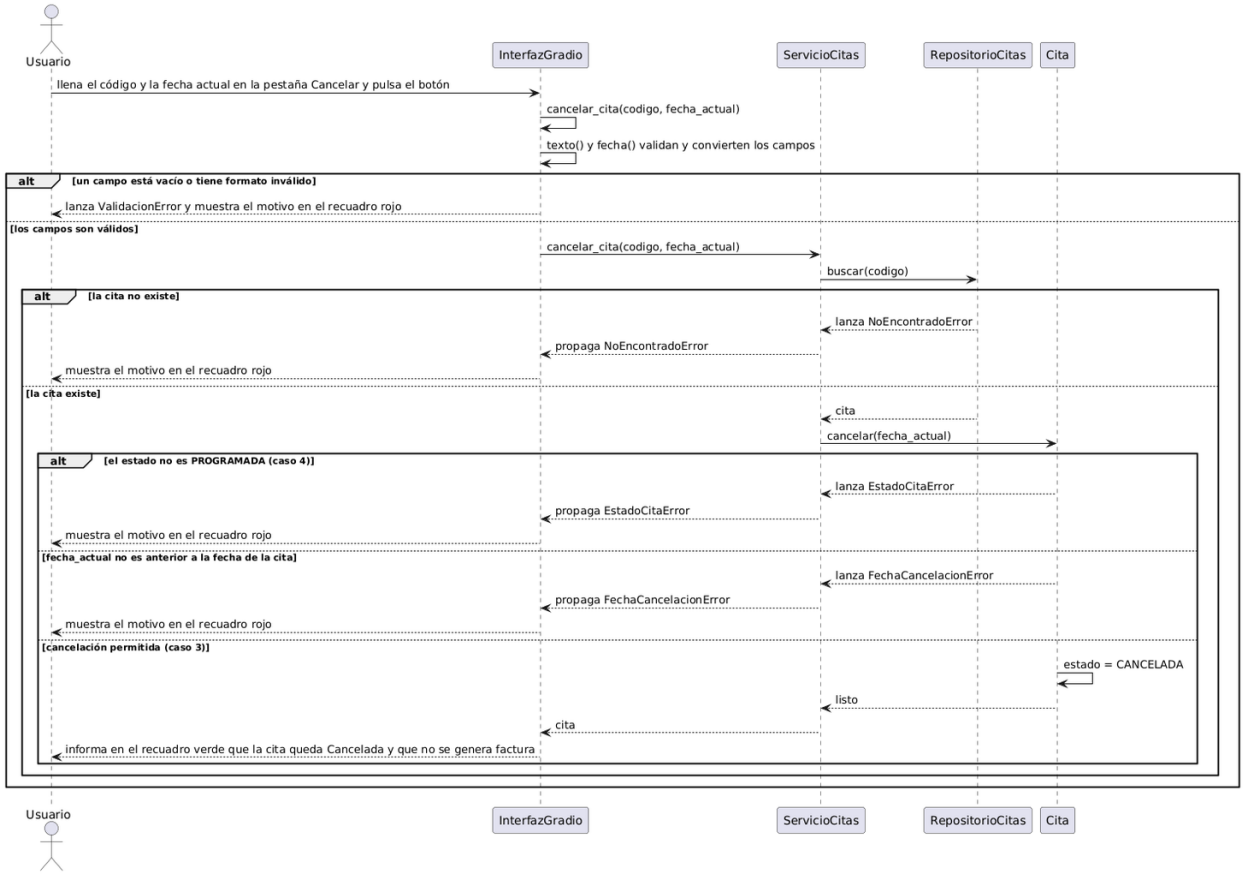
Arranque de la aplicación



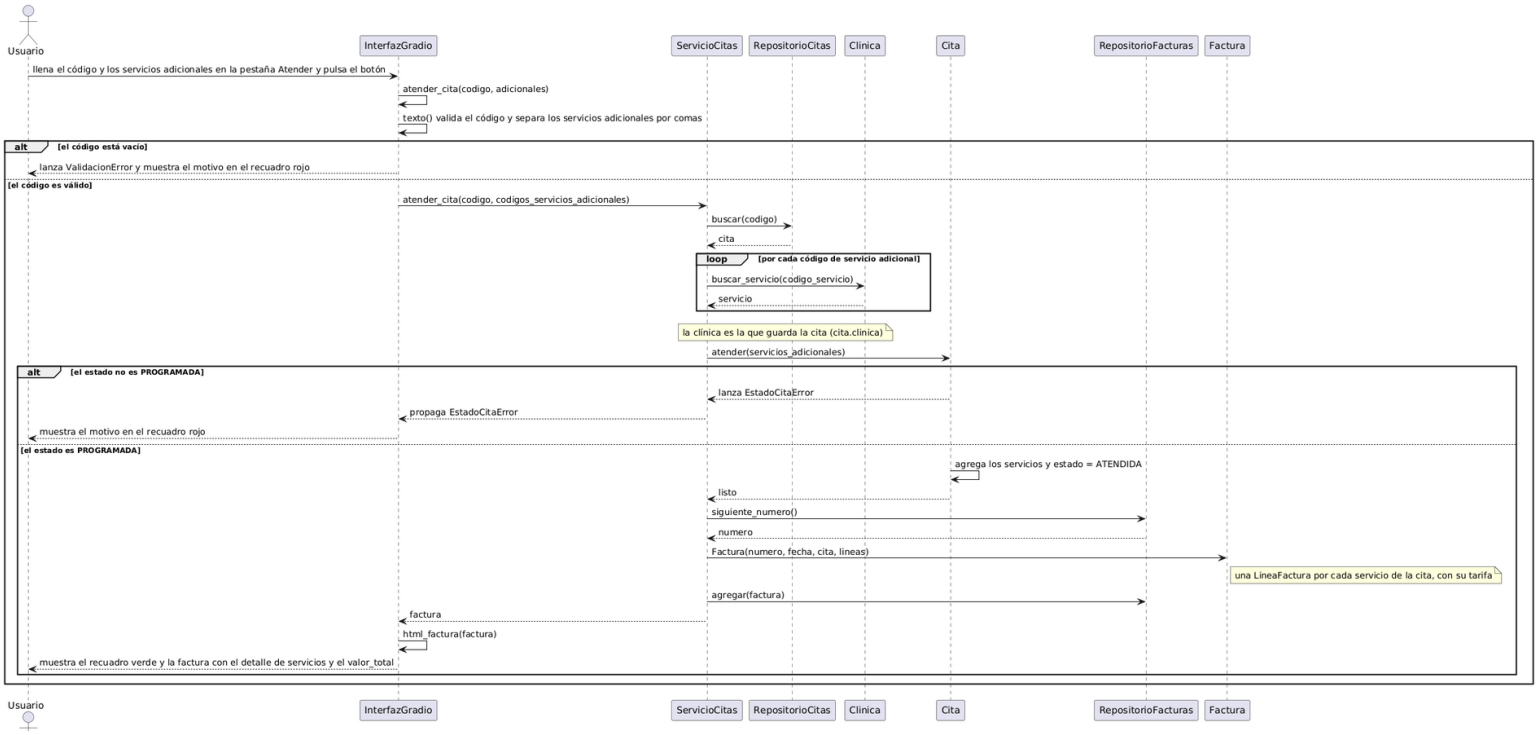
Programar una cita


Usuario
InterfazGradio
ServicioCitas
RepositorioClinicas
Clinica
Propietario
RepositorioCitas
Cita
llena los campos de la pestaña Programar y pulsa el botón
programar_cita(nit, cedula_propietario, mascota, cedula_veterinario, servicio, fecha, hora)
texto(), fecha() y hora() validan y convierten cada campo
alt
[un campo está vacío o tiene formato inválido]
lanza ValidacionError y muestra el motivo en el recuadro rojo
[los campos son válidos]
programar_cita(nit, cedula_propietario, nombre_mascota, cedula_veterinario, codigo_servicio, fecha, hora)
buscar(nit)
clinica
buscar_propietario(cedula_propietario)
propietario
buscar_mascota(nombre_mascota)
mascota
buscar_veterinario(cedula_veterinario)
veterinario
buscar_servicio(codigo_servicio)
servicio
una búsqueda sin resultado lanza NoEncontradoError
buscar_programada_veterinario(nit, cedula_veterinario, fecha, hora)
cita o None
alt
[el veterinario ya tiene una cita programada (caso 2)]
lanza ConflictoAgendaError
muestra el motivo en el recuadro rojo
[el veterinario está libre]
buscar_programada_mascota(nit, cedula_propietario, nombre_mascota, fecha, hora)
cita o None
alt
[la mascota ya tiene una cita programada]
lanza ConflictoAgendaError
muestra el motivo en el recuadro rojo
[la mascota está libre]
siguiente_codigo()
codigo
Cita(codigo, fecha, hora, clinica, propietario, mascota, veterinario, servicio)
estado inicial PROGRAMADA
agregar(cita)
cita
muestra el código de la cita en el recuadro verde
Usuario
InterfazGradio
ServicioCitas
RepositorioClinicas
Clinica
Propietario
RepositorioCitas
Cita

Cancelar una cita



Atender una cita y generar la factura


Usuario
InterfazGradio
ServicioCitas
RepositorioCitas
Clinica
Cita
RepositorioFacturas
Factura
llena el código y los servicios adicionales en la pestaña Atender y pulsa el botón
atender_cita(codigo, adicionales)
texto() valida el código y separa los servicios adicionales por comas
alt [el código está vacío]
lanza ValidacionError y muestra el motivo en el recuadro rojo
[el código es válido]
atender_cita(codigo, codigos_servicios_adicionales)
buscar(codigo)
cita
loop [por cada código de servicio adicional]
buscar_servicio(codigo_servicio)
servicio
la clínica es la que guarda la cita (cita.clinica)
atender(servicios_adicionales)
alt [el estado no es PROGRAMADA]
lanza EstadoCitaError
propaga EstadoCitaError
muestra el motivo en el recuadro rojo
[el estado es PROGRAMADA]
agrega los servicios y estado = ATENDIDA
listo
siguiente_numero()
numero
Factura(numero, fecha, cita, lineas)
una LineaFactura por cada servicio de la cita, con su tarifa
agregar(factura)
factura
html_factura(factura)
muestra el recuadro verde y la factura con el detalle de servicios y el valor_total